

Convolutional Neural Networks (CNNs)

COMPLETE ARCHITECTURE STUDY GUIDE: FEATURE EXTRACTION TO CLASSIFICATION

1. CNN vs. Human Vision: Processing Paradigms

Traditional Artificial Neural Networks (ANNs) process images by flattening the pixel matrix into a single continuous 1D vector. This catastrophic restructuring completely discards **spatial topology**—the localized pixel relationships that form edges, textures, and shapes.

Human Vision vs. CNNs: Human vision processes scenes hierarchically. Light hits the retina, and the primary visual cortex (V1) fires to detect basic local orientations and edges. Higher cortical regions assemble these primitives into complex objects (faces, objects). Convolutional Neural Networks mimic this by passing small, localized **receptive fields** across an image, capturing spatial context and building a hierarchical representation of features without requiring parameter-heavy full connectivity at every layer.

2. CNN Convolution: Feature Detection & Feature Maps

The core computational operational block of a CNN is the **Convolution Layer**. Instead of mapping a separate weight to every single pixel, a small matrix called a **Kernel (or Filter)** slides across the input image dimensions. At each step, a mathematical *dot product* is performed between the kernel weights and the covered receptive field pixels. The scalar results are recorded in a new matrix called a **Feature Map (or Activation Map)**.

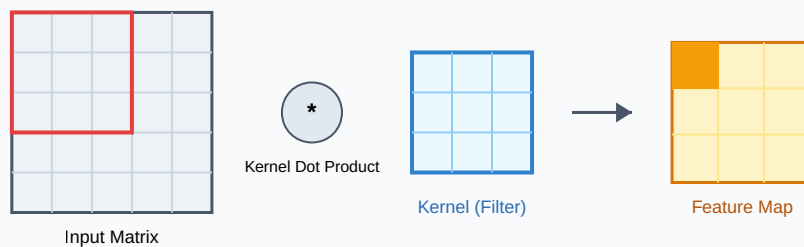


FIGURE 1: CONVOLUTION OPERATION MATRIX MAPPING

MATHEMATICAL FORMULA FOR OUTPUT DIMENSIONS

The structural spatial dimensions of an output feature map are determined by the input shape (W), kernel size (K), padding allocation (P), and stride frequency (S):

$$O = \lfloor (W - K + 2P) / S \rfloor + 1$$

Where **Padding** (P) adds zero-valued borders around the input image to preserve spatial margins, and **Stride** (S) defines how many pixel units the kernel jumps during its scan.

3. ReLU in CNN: Injecting Non-Linearity

Every convolution step executes purely linear operations (additions and multiplications). Real-world visual structural features are highly non-linear. Therefore, immediately following a convolution layer, every scalar value in the feature map passes through an activation function, typically the **Rectified Linear Unit (ReLU)**.

RELU FORMULATION

$$f(x) = \max(0, x)$$

ReLU replaces all negative pixel values in the feature maps with an absolute zero value, while preserving positive metrics intact. This clamps down negative trends, highlighting dominant feature activations and boosting computational efficiency.

4. Step 2 — Max Pooling in CNN: Spatial Reduction & Regularization

To reduce computational strain, downsample feature metrics, and prevent overfitting, the network uses a **Pooling Layer** (most commonly **Max Pooling**). Like convolution, a pooling window slides across the feature map, but instead of calculating a dot product, it simply extracts the maximum value within that window.

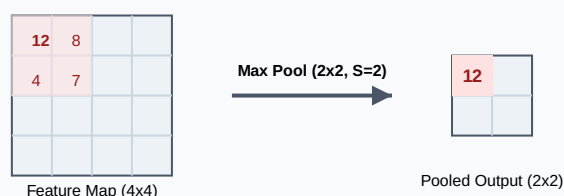


FIGURE 2: MAX POOLING (2X2 WINDOWS WITH STRIDE=2)

- **Spatial Invariance:** Max pooling helps the model detect features regardless of their exact position, tilt, or lighting conditions in the image.

- **Dimensionality Reduction:** It discards up to 75% of less-activated pixel data, focusing computational resources purely on the most critical visual cues.

5. Step 3 — CNN Feature Map Flattening: Bridging the Divide

After multi-stage cycles of Convolution, ReLU, and Max Pooling, the output is a set of multi-channeled 3D feature matrices. To transition from feature detection to class prediction, these arrays must be converted into a format acceptable by classic dense classifier structures. This stage is known as **Flattening**.

The flattening step unwraps the 3D tensor volume (Height × Width × Channels) row by row, reformatting it into a single long **1D Feature Vector**. This vector contains all the high-level features extracted by the convolutional layers, ready to serve as input for the final classification layers.

6. Step 4 — CNN Classification: Fully Connected (FC) Layer Processing

The flattened 1D vector feeds into one or more **Fully Connected (FC) Layers** (identical to standard multi-layer perceptron networks). Here, every input node connects to every neuron in the subsequent layer, with each link managed by a trainable weight parameter.

While the early convolutional layers focus on *extracting local features*, the FC layers focus on *global feature combination*. They analyze the spatial configurations within the 1D vector to determine which patterns correspond to specific target classes (e.g., assessing if a mix of detected "wheels," "windshield," and "headlights" indicates a car).

7. Softmax & Cross-Entropy Loss Functions

The final output layer of a classification network produces raw, unconstrained numerical scores called **Logits**. To turn these scores into clear, actionable classifications, the model applies the **Softmax** function alongside **Cross-Entropy Loss**.

SOFTMAX MATHEMATICAL FUNCTION

Softmax normalizes the raw logit scores (z_i) for all target classes into a valid probability distribution where each class probability falls between 0 and 1, and all class probabilities sum to exactly 1.

$$P(y = i | x) = e^{z_i} / \sum_{j=1}^C e^{z_j}$$

Where C represents the total number of distinct target classification categories.

CATEGORICAL CROSS-ENTROPY LOSS EQUATION

Cross-Entropy measures the performance of the model by penalizing the divergence between the predicted probability distribution (p_i) and the actual true class distribution (y_i , typically a one-hot encoded vector):

$$L = - \sum_{i=1}^C y_i \log(p_i)$$

If the model assigns a low probability to the correct class, Cross-Entropy heavily penalizes the system with a large loss value, forcing the optimizer to adjust weights during backpropagation.

8. CNN Architecture Summary: The End-to-End Pipeline

Pipeline Stage	Core Mathematical Operational Impact	Data Format Evolution
Input Stage	Stores raw visual pixel attributes across color dimensions.	3D Tensor: $[H \times W \times \text{Channels}]$
Convolution + ReLU	Extracts localized features using sliding kernels and drops negative activations to inject non-linearity.	3D Tensor: $[H' \times W' \times \text{ext}\{\text{Kernels}\}]$
Max Pooling	Downsamples feature maps to reduce spatial size and prevent overfitting.	3D Tensor: $[H'' \times W'' \times \text{ext}\{\text{Kernels}\}]$
Flattening Stage	Unwraps multi-channel 3D feature arrays into a linear configuration.	1D Vector: $[(H'' \times W'' \times \text{ext}\{\text{Kernels}\}) \times 1]$
Fully Connected Layer	Learns global feature combinations across all extracted traits.	1D Logits Vector: $[C \times 1]$
Softmax Classifier	Normalizes raw logit outputs into clear class probabilities.	Normalized Probability Distribution